Backend and Database Workflow

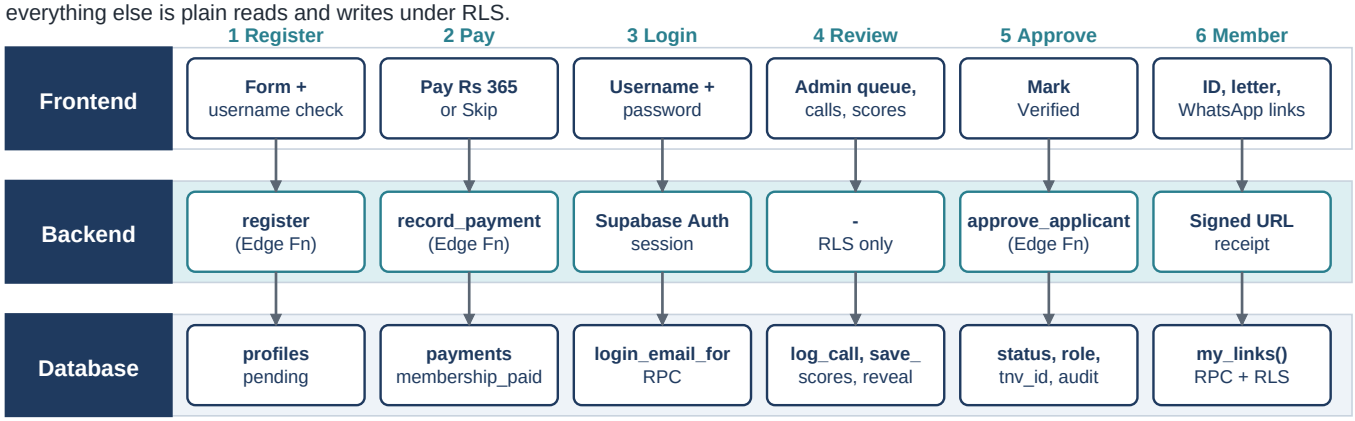
TNV Hub Portal, Phase 1. One page of stack and flow, one page of detail.

Tech stack

Layer	Technology	Used for
Platform	**Supabase**	Hosts everything below. Single project, one service key kept in secrets.
Database	**PostgreSQL** (SQL, PL/pgSQL)	Tables, constraints, RPC functions, views, sequences, row-level security (RLS).
Auth	**Supabase Auth**	Email and password. Username is resolved to email by a database function.
Server logic	**Supabase Edge Functions** (TypeScript on Deno)	Privileged actions with the service key: register, record_payment, approve_applicant, create_donation.
Files	**Supabase Storage**	Private bucket for receipts and photos. Signed URLs only.
Email	**Resend or SMTP**	Welcome and approval emails. Password reset uses Supabase built-in email.
Payments	**Payment gateway or mock**	Fixed Rs 365 membership, donations from Rs 1.
Client	**React + supabase-js**	Talks to Supabase directly. RLS is the access-control layer.

End-to-end workflow

Each stage runs top to bottom: the user action, the backend step, then the database change. Edge Functions use the service key; everything else is plain reads and writes under RLS.



Stage by stage: who does what

Stage	Backend (Edge Functions, email, storage)	Database (tables, RPCs, RLS)
1 Register	**register**: validate, reject duplicates, create auth user, encrypt Aadhaar, send welcome email (no password), return session.	**username_available**. New **profiles** row (pending), **notifications** row. Format checks on username and phone.
2 Pay	**record_payment**: verify payment, fix amount at Rs 365, build receipt PDF into the private bucket.	**payments** row, **membership_paid** = true, receipt_no from a sequence. Failure changes nothing else.
3 Login	Supabase Auth signs in with email and password. Same generic error for wrong user or wrong password.	**login_email_for(username)** returns the email or nothing.
4 Review	None. Admin screens read straight through RLS.	Admin policies read all profiles. Aadhaar masked. **reveal_aadhaar** (Master Admin) writes **audit_log**.
5 Calls and scores	None.	**log_call** (scheduled needs a time). **save_scores** (only 0, 0.5, 1).
6 Approve	**approve_applicant**: check admin, all active questions scored, sum, generate TNV ID, commit, send approval email.	Updates status, role, tnv_id. Inserts **verifications** and **audit_log**. No way back to pending.
7 Member access	Signed URL for receipt download.	**my_links()** and RLS on **whatsapp_links**: pending users get zero rows.
8 Donate (signed out)	**create_donation**: validate Rs 1 or more, take payment, insert with the service key.	**donations** table. Admins read only. No public insert policy.

Backend workflow (Edge Functions)

Four functions cross trust boundaries. Each one verifies the caller itself and never trusts a user id, role or amount from the request.

register public

1. Validate: username 4 to 20 chars, phone 10 digits (6 to 9 start), Aadhaar 12 digits, district exists.
2. Reject duplicate username or email with a clear message.
3. Create the auth user with the applicant's own password.
4. Encrypt Aadhaar. Keep encrypted value and last 4 digits.
5. Insert profile (pending, unpaid) and the admin notification.
6. Send welcome email: username and login link only.
7. Return the session. On failure after step 3, delete the auth user.

record_payment signed-in user

1. Read the user id from the verified token, not the body.
2. Verify the result with the gateway (or the mock marker).
3. Amount is fixed at Rs 365 by the server.
4. If failed: change nothing else. Registration is unaffected.
5. If paid: receipt_no TNV-R-year-number from a sequence; insert payments row; set membership_paid = true.
6. Build the receipt PDF into the private bucket.
7. Return receipt_no and a signed URL. Same gateway_ref twice returns the same receipt.

approve_applicant admin only

1. Verify the caller is master_admin or dept_admin.
2. Reject if the applicant is missing or already approved.
3. Refuse if any active question has no score.
4. Sum the scores and generate the TNV ID from a sequence.
5. In one transaction: set status, role, tnv_id, insert verifications.
6. Write the audit_log row.
7. Send the approval email. Email failure is logged, not rolled back.

create_donation public

1. Validate: whole number, Rs 1 or more. Limit name and message length.
2. Take payment through the gateway (or the mock).
3. Confirm success on the server before writing.
4. Insert into donations with the service key.
5. Anonymous donor: store donor_name as null.
6. Return a plain confirmation. Never return other donations.

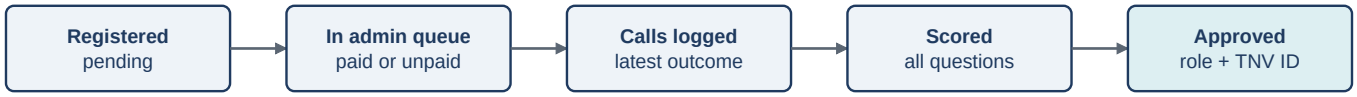
Database workflow

Create in order: types, reference tables, profiles + is_admin(), then the rest. Names are final for Phase 1: add columns, never rename.

Database functions (RPCs)

Function	Caller	What it does	Rule enforced
username_available(u)	Anyone	True if the username is free	Returns only a boolean
login_email_for(username)	Anyone	Returns the email for sign-in	Nothing for unknown names. Rate limit it.
log_call(...)	Admin	Inserts a call_logs row	scheduled requires callback_at. Unlimited attempts.
save_scores(...)	Admin	Upserts scores per question	Score must be 0, 0.5 or 1. Question must be active.
reveal_aadhaar(id)	Master Admin	Returns the decrypted Aadhaar	Writes audit_log in the same call.
my_links()	Approved user	Own district, wing, 13 sectors, announcements, social	Zero rows while pending.

Access control (RLS) and protections

Rule	How it is enforced
RLS on every table; no policy means no rows	Enable RLS everywhere. Applicants get no policy on call_logs, interview tables, verifications, audit_log, notifications.
Applicant reads only own data	profiles and payments: select own row. whatsapp_links: only when status = approved.
Applicant cannot self-approve	RLS cannot protect columns. Use column-level UPDATE grants or a BEFORE UPDATE trigger on status, role, tnv_id, wing_id, membership_paid and Aadhaar columns.
Aadhaar never reaches the browser	Clients read profiles_public (no Aadhaar columns). Full number only through reveal_aadhaar.
Admin checks, secrets, recovery	is_admin() in policies and RPCs; master-only actions check master_admin. Service key only in Edge Function secrets. Backups on before real data.

Status lifecycle